

Gossip-based Aggregation in Large Dynamic Networks

realizat de Tudor Apostol și Victor Buscoveanu

Context și definirea problemei

1. Context

- Rețele distribuite masive (ex: clustere de servere) unde nu există un coordonator central
- Necesitatea de a agrega date (ex: calcularea mediei temperaturilor sau încărcării CPU)

2. Problema

- Cum calculăm media globală când niciun nod nu cunoaște toți participanții?

3. Soluția aleasă

- Protocol Gossip: Diseminarea informației prin schimburi locale, aleatoare și repetate între perechi de noduri

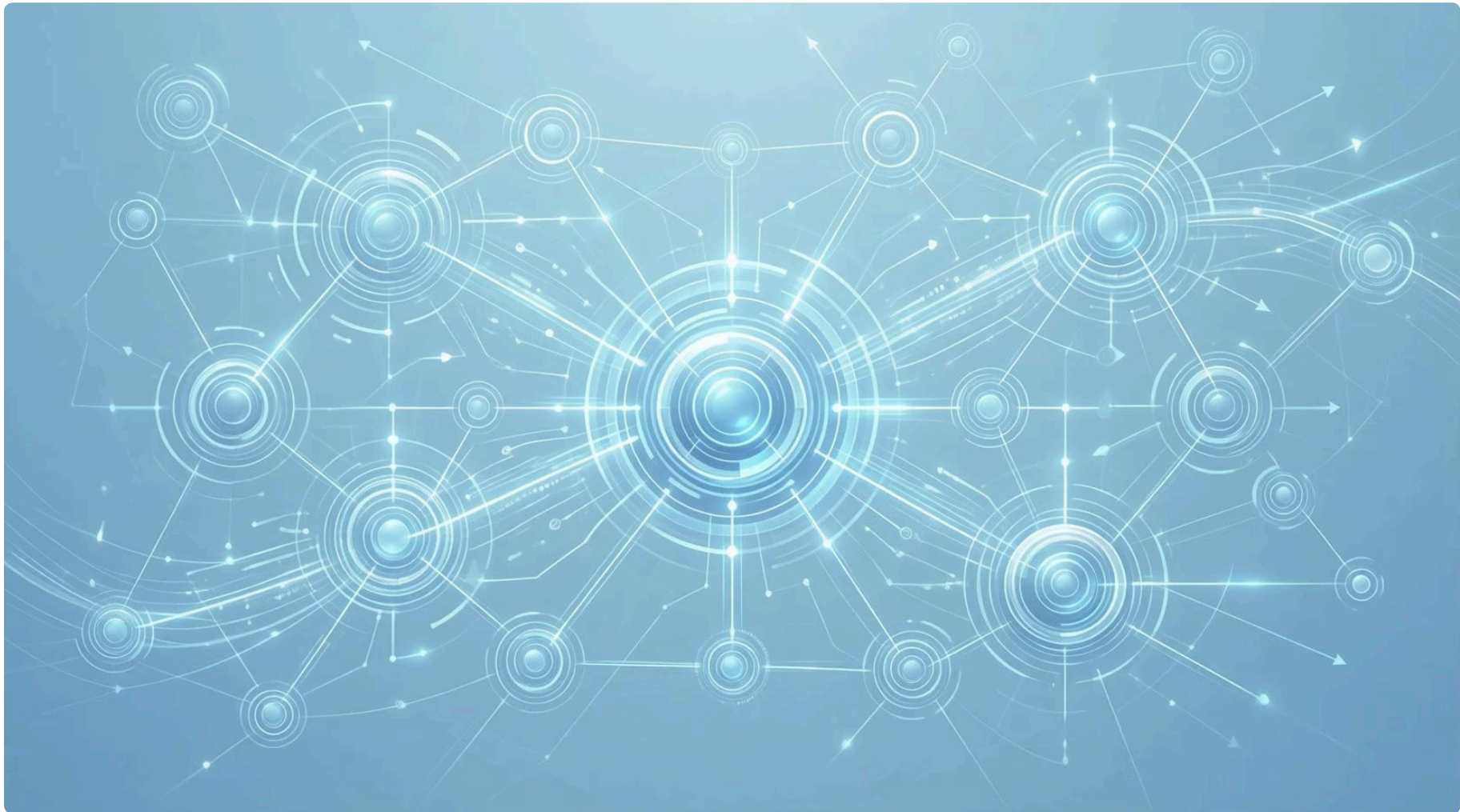
Algoritmul: de la centralizat la decentralizat

Abordarea centralizată

- Un proces central citește toate valorile și calculează media
- Problemă: Single Point of Failure, nu scalează la 10.000+ noduri

Abordarea decentralizată

- Pairwise Averaging: Două noduri își actualizează simultan valoarea
- Prin repetare, varianța scade și sistemul converge spre media globală



Arhitectura SPMD în Go

Model SPMD (Single Program, Multiple Data)

- Toate nodurile rulează aceeași funcție `run()`, dar pe date diferite

Structura Node

- Stare: Value (date), Alive (status activ/inactiv)
- Concurență: `sync.Mutex` pentru protecția datelor (Thread-Safety)
- Identitate: Id unic pentru rezolvarea conflictelor

Execuție

- Fiecare nod rulează în propria Goroutină
- Comunicarea este simulată prin acces direct la memorie protejat de mutex-uri



Mecanisme cheie de implementare

Scalabilitate ($O(1)$)

- Funcția `pickRandomNeighbor` alege un vecin aleator direct prin index, fără a parcurge toată lista
- Permite rularea cu `NumNodes = 10000` fără degradarea performanței

Prevenirea Deadlock-ului

- În funcția `Gossip`, lock-urile se fac ordonat crescător după `Id`: `if n1.Id > n2.Id { first, second = n2, n1 }`
- Elimină riscul de "Circular Wait"

Convergență

- Funcția `valuesConverged` verifică stabilitatea sistemului folosind o toleranță `ConstTolerance = 0.0001`



Dinamica Rețelei (Fault Tolerance)

01

Simularea Eșecurilor:

- Funcția RandomToggle oprește și repornește noduri aleatoriu

02

Comportament la revenire:

- Când un nod revine online (ToggleStatus), valoarea sa este resetată ($n.Value = Id * 10.0$)
- Efect: Introduce o perturbare în sistem, forțând algoritmul să re-converga la o nouă medie

03

Rezultat:

- Demonstrează capacitatea de a se redresa a algoritmului



Analiza Complexității

1

Timp de Convergență:

- Teoretic: $O(\log N)$ – convergență exponențială
- Practic: Sistemul se stabilizează rapid chiar și la 10.000 de noduri

2

Complexitate Mesaje:

- Liniară ($O(N)$) per runda de gossip a rețelei

3

Corectitudine:

- Suma globală este invariantă (suma a două noduri înainte de gossip = suma după gossip)
- Media rămâne constantă (în absența eșecurilor)



Concluzii și demonstrație

Sinteză

- S-a implementat un sistem robust, thread-safe în Go
- Codul gestionează corect concurența (mutex) și eșecurile (RandomToggle)

Rezultate

- Sistemul atinge starea `>>> SYSTEM STABLE <<<` chiar și sub stresul modificărilor dinamice

